

IT Infrastructure Security Assessment

BACC Airport Operations Portal

Philip S.W. Goldson International Airport (PGIA)

Assessment date: 31 August 2026 · Version 1.0

Prepared for: Belize Airports Authority Company (BACC)

Prepared by: Vision Forge Ltd

Scope. This document assesses the security posture of the BACC operations portal as deployed today (GitHub Pages static demo with client-side mock data) and outlines requirements for a production deployment suitable for airport authority operations.

1. Executive summary

The portal is currently a demonstration environment published as a static site on GitHub Pages. It is appropriate for showcasing approved forms, workflows, and UI to BACC stakeholders. It is not production-ready for handling live operational data, staff credentials, or safety-critical records without substantial infrastructure and security hardening.

Supabase migrations and Row Level Security (RLS) policies exist in the repository and represent a sound foundation for production. The live demo bypasses these controls by storing all data in browser localStorage with passwordless demo sign-in.

Severity	Count	Summary
Critical	3	No real authentication; client-tamperable data; unauthenticated PDF APIs
High	4	Client-only role checks; sensitive data in browser; PII on public demo; missing headers
Medium	5	Incomplete RLS mapping; upload validation; audit gaps; PWA cache; location retention
Low	3	Supply chain; verbose API errors; demo session handling

2. System architecture

2.1 Application stack

- Frontend: React 19, Vite 7, Tailwind 4, React Router 7, PWA (Workbox)
- Data (demo): Mock repositories persisting to localStorage (bacc-demo-store)
- Data (production): Supabase PostgreSQL + Auth + Storage with RLS in supabase/migrations/
- PDF export: Node.js API routes overlaying values onto approved base PDFs via pdf-lib
- Deployment (demo): GitHub Pages (gh-pages branch) — static files only; no server APIs

2.2 Authentication (demo mode)

- Password is not validated when Supabase is not configured
- User ID stored in sessionStorage (bacc-local-auth)
- Any visitor can sign in as any seeded demo account
- AppShell route guard only checks that a session exists — not credentials

2.3 Authorisation

Role-based access (OM, duty manager, apron supervisor, electrical tech, SMS, etc.) is enforced primarily in React components and the mock repository. Supabase RLS policies restrict submissions to owners and OM/admin roles, but these policies are inactive while the demo data source is in use.

3. Risk register — Critical

Risk	Recommendation
C1 — No real authentication	Enable Supabase Auth or enterprise IdP. Enforce password policy, lockout, and MFA for OM/admin. Remove demo bypass in production.
C2 — Client-tamperable data	Set VITE_DATA_SOURCE=supabase. Treat browser storage as offline cache only. All writes via authenticated API.
C3 — Unauthenticated PDF APIs	Require JWT on every /api/export-* route. Validate caller owns the submission. Rate-limit and audit exports.

3. Risk register — High

Risk	Recommendation
H1 — Role checks client-side only	Enforce all role decisions in Supabase RLS and API middleware. Client checks are UX hints only.

H2 — Sensitive data in browser storage	Store artifacts in Supabase Storage with signed URLs. Encrypt at rest. Clear cache on sign-out.
H3 — PII on public demo	Use synthetic data on public demos. Production must be private. Obtain staff consent for go-live.
H4 — Missing security headers	Configure CSP, X-Frame-Options: DENY, Strict-Transport-Security, Referrer-Policy at CDN/proxy.

3. Risk register — Medium & Low

Risk	Recommendation
M1 — Incomplete RLS role mapping	Complete RLS per BACC permission matrix (A3/A4). Pen test before go-live.
M2 — File upload without server validation	Validate type, size, dimensions server-side. Consider AV scanning.
M3 — Audit trail gaps in demo	Immutable audit log on every submission, acknowledgment, and incident transition.
M4 — PWA service worker caching	Never cache authenticated API responses. Version cache on deploy.
M5 — Location data retention	Define retention period. Restrict map providers. Document lawful basis.
L1 — Dependency supply chain	Enable Dependabot. Run npm audit in CI. Pin lockfile.
L2 — Verbose API errors	Generic errors to clients; log details server-side only.

4. Data classification

Data type	Sensitivity	Production requirement
Staff directory	Internal / PII	RLS-protected profiles; no public exposure
Inspection submissions	Operational / safety	PostgreSQL with RLS; immutable after submit (BACC §11)
Digital signatures	Legal / evidential	Encrypted storage; bind to user and timestamp
Incident photos & GPS	Operational / PII	Private object storage; signed URLs; retention policy
Exported PDFs	Official records	Authenticated API only; audit each export

5. Production infrastructure checklist

- Hosting: Private deployment behind HTTPS — not public GitHub Pages
- Database: Supabase Pro with RLS, daily backups, point-in-time recovery
- Authentication: Supabase Auth, session timeout, MFA for privileged roles
- API layer: JWT validation on every request; rate limiting per user/IP
- Object storage: Supabase Storage with bucket policies; no public evidence buckets
- Network: WAF (Cloudflare / Azure Front Door); optional IP allowlisting for admin
- Monitoring: Error tracking, auth failure alerts, weekly audit log review
- Compliance: Data processing agreement; assess Belize data residency
- Penetration testing: Third-party assessment before live operational data
- Disaster recovery: Documented RTO/RPO; tested restore annually

6. Deployment comparison

Capability	GitHub Pages (demo)	Production target
------------	---------------------	-------------------

Authentication	None (demo picker)	Supabase Auth + MFA
Data persistence	Browser localStorage	PostgreSQL with RLS
PDF overlay export	Not available	Authenticated API routes
Audit logging	None in demo	Immutable server-side audit trail
Security headers	GitHub defaults	CSP, HSTS, frame denial

7. Recommended phases

Phase 1 — Before any live data (4–6 weeks)

- Deploy Supabase with migrations and RLS enabled
- Switch production to VITE_DATA_SOURCE=supabase
- Implement JWT validation on PDF export APIs
- Remove real staff PII from public demo
- Configure security headers at hosting layer

Phase 2 — Pre go-live (2–4 weeks)

- Complete role permission matrix in RLS policies
- Enable MFA for OM, COO, and admin accounts
- Server-side file upload validation and immutable audit logging
- Internal security review and dependency audit

Phase 3 — Go-live and ongoing

- Third-party penetration test
- Security monitoring and incident response runbook
- Quarterly access review; annual disaster recovery drill

8. Conclusion

The BACC operations portal demonstrates a well-structured application architecture with approved-form fidelity, incident workflows, and a credible path to production via Supabase. The current GitHub Pages deployment is intentionally lightweight and must be treated as a non-production demo.

Moving to production requires authenticating every user, enforcing authorisation server-side, protecting PDF generation APIs, and hosting on infrastructure with appropriate monitoring, backup, and security controls.

Document control: BACC Airport Portal Security Assessment v1.0 · 31 August 2026. Classification: Internal — BACC / Vision Forge Ltd.